

ROCKWELL METALS

Platform Flow Audit

End-to-end review of the customer and staff journeys — what is broken, what is placeholder, what is slow, and what falls short of production standards.

SCOPE	100 route & component files · 14,562 LOC TSX/TS · 6 legacy DOM runtimes · 5 API routes · 25,457-product catalog
REALMS	Customer storefront & vault · Staff operations console
FINDINGS	47 across 5 categories — 17 critical
METHOD	Static read of every route, the domain layer, the API surface, and the catalog data. No runtime execution — dependencies are not installed.

01 Executive summary

Rockwell is two applications wearing one skin. Underneath the storefront there is a genuinely well-built commerce system: a typed domain model, an eight-state order machine, serial-level vault custody, role-scoped staff tooling, and an append-only audit ledger that records every transition. That part is thoughtfully designed and largely works.

Sitting on top of it is the original static prototype — the homepage, the market floor, the product page and the vault dashboard — which is hardcoded HTML driven by ES5 DOM scripts. It shares no data with the system underneath. **The pages a customer actually lands on are the prototype; the pages that hold the real logic are mostly reached only by deep link.** The result is a journey that looks complete and breaks the moment you follow it end to end.

Separately, and more urgently: there is no authentication or authorization anywhere in the stack. Sign-in accepts an email address and nothing else, sessions are unsigned localStorage JSON that carries the user's role, and the single API route exposes the entire database — every customer's identity documents status, address, holdings and serial numbers — to any unauthenticated GET.

17 CRITICAL (P0)	20 HIGH (P1)	9 MEDIUM (P2)	1 LOW (P3)
--	------------------------------------	-------------------------------------	----------------------------------

Findings by category

CATEGORY	COUNT	WHAT IT MEANS	SECTION
Security & access control	10	No auth, no authz, full data exposure, unauthenticated writes to disk	§05
Broken behaviour	9	Flows that dead-end, filters that return nothing, purchases that vanish	§06
Placeholder / filler	10	Fabricated reviews, fake tax forms, dead legal links, theatre auth	§07
Performance	9	78 MB catalog in the client bundle, 8 MB HTML pages, whole-DB polling	§08
Standards & quality	9	No error boundaries, no SEO surface, modal a11y, zero tests	§09

The single most consequential finding

Every product image on the site — all 25,457 — still points at [sdbullion.com](#) (13,034) and [images-apmex.com](#) (12,411). The ingestion pipeline described in the handover document has never run: its state database holds zero rows and the link map reports **25,457 unmatched**. The storefront is currently serving two competitors' CDNs on every page, which is both a hard availability dependency and the exact exposure the pipeline was written to close.

02 The two-track problem

Sixteen surfaces read live data through `useRm / /api/rm`. Five are static prototypes driven by ES5 DOM scripts in `app/lib/*.js`. The split does not follow any product boundary — it follows build order, and it cuts straight through the middle of the customer journey.

ROUTE	IMPLEMENTATION	DATA SOURCE	STATE
/ (home)	Static prototype	Hardcoded markup + home.js	Demo only
/market	Static prototype	14 hardcoded tiles + market.js	Demo only
/product	Static prototype	One hardcoded Gold Buffalo	Demo only
/vault	Static prototype	Hardcoded \$28,098 + vault.js	Demo only
/gold /silver /platinum	Server-rendered	products.json (25,457)	Live
/best-sellers /new-arrivals /on-sale	Server-rendered	products.json filters	Live
/product/[id]	Server-rendered	products.json + seeded fill	Live (partly seeded)
/auth/*	Client + API	/api/rm	Live
/checkout/[lock_id]	Client + API	localStorage lock + /api/rm	Live
/orders /orders/[id] /tracking	Client + API	/api/rm	Live
/vault/holdings /sell-back /delivery	Client + API	/api/rm	Live
/vault/vaultplan /rewards	Client + API	/api/rm	Live
/tax-center /support /drops	Client + API	/api/rm	Live
/admin/* (17 routes)	Client + API	/api/rm	Live

Why this is the root cause, not a cosmetic issue

The nav sends customers to `/market`, where all fourteen tiles link to `/product` — one fixed demo page. The 25,457 real products are reachable only from `/gold`, `/silver` and `/platinum`. A customer who does find a real product and buys it is redirected to `/vault`, which is hardcoded and will never show their purchase. Three of the nine findings in §06 are downstream of this one structural decision.

03 Customer journey, step by step

#	STEP	WHAT HAPPENS TODAY	VERDICT
1	Land on /	Hardcoded hero, hardcoded \$2,387.40 spot, six demo coin cards	Prototype
2	Open /market	14 demo tiles; Weight filter returns zero results; every tile links to /product	Broken
3	Browse /gold, /silver	Real catalog, but all tiles render into one document and the page is banded 'Placeholder inventory'	Live, flawed
4	Open a product	Real data + fabricated rating, buyer count and three invented reviews; gallery shows one image four times	Live, filler
5	Sign up	Name / email / phone, no password. Account is created immediately	No auth
6	Verify identity	3.6s scripted animation, then Tier 2 CLEARED written server-side. No document is uploaded	Theatre
7	Price lock	120s timer works; the locked spot is the literal 2387.4, refresh applies a random $\pm 0.3\%$ drift	Live, fake spot
8	Checkout	Real: custody, rail, surcharge, volume tiers, placeOrder. QR code is decorative art	Live, works
9	Order confirmed	Order lands in the store, audit entry written, status machine engaged	Live, works
10	Open /vault	Static \$28,098 / 11 oz. The purchase from step 9 is nowhere	Broken
11	'Buy more gold' in vault	Opens OrderModal — a second, parallel checkout that persists nothing	Broken
12	/orders	Correct and live. Signed out, it lists every user's orders	Live, leaks
13	Vault sub-pages	Passport, sell-back, withdrawal, DCA, rewards all real and well-built	Live, works
14	/tax-center	Real ledger; cost basis is proceeds x 0.955 and gain a flat 4.5% on every disposal	Live, fabricated

Steps 8, 9, 12 and 13 are the strongest work in the codebase — the settlement flow, the order state machine and the custody passports are well-modelled and correct. The journey fails at the joins: step 2 never reaches the catalog, step 10 never reflects step 9, and step 11 offers a second checkout that writes nothing.

04 Staff & operations journey

The ops console is the healthiest part of the platform. All seventeen routes read live data, every mutation goes through the same audited transition layer, and the role model is genuinely thought through. The problems here are enforcement and ergonomics, not correctness.

STAGE	TOOLING	STATE
Access	Five demo roles selectable from the gate; /auth/demo mints any of them by URL	Unenforced
Command centre	Live queue counts across orders, fulfilment, compliance, sell-backs, OTC	Works
Orders · OMS	Grid + inspector; confirmPayment, startAssay, allocate, dispatch, cancel all wired	Works
Serial allocation	Binds serials to line items, decrements lots, mints vault passports	Works
Pick / pack / ship	Scan station rejects serials that do not match the order's allocation	Works
Transit control	Five-checkpoint route, exception flagging, delivery signature capture	Works
Sell-back desk	Approve / reject / disburse with KYC status shown inline	Works
Customer 360	GMV, vault oz, risk, freeze, staff notes	Works
Compliance / KYC	Tier and status override, writes before/after to the ledger	Works
Pricing engine	Premiums, tiers, surcharges, spread — preview is computed off hardcoded 2387.4	Works, fake spot
Support & OTC	Ticket threads, assignment, priority, quote generation	Works
Audit ledger	Append-only, before/after JSON, capped at 500 entries	Works, capped
Catalog Studio	2,110-line client; drives three unauthenticated API routes, two of which spawn Python	Risky

The gap between the model and the enforcement

OpsShell defines a careful role matrix and hides tools a role should not see. None of it is enforced. There is no `middleware.ts`; the gate is a React early-return, the role it reads comes from `localStorage`, and the API it protects accepts any action from anyone. The design is right — it has simply never been moved to the server.

05 Security & access control

These are listed first because they gate everything else: no other fix matters while the API is open. All ten are in the application's own code, not in dependencies.

P0 S-01 The entire database is readable without authentication

WHERE `app/api/rm/route.ts:11`

IMPACT **GET /api/rm** returns the whole *RmDb* — every user's email, phone, KYC tier and status, risk rating and private staff notes; every order with its delivery address; every vault holding with its serial number and bay location; every shipment, sell-back, support thread and audit entry. One unauthenticated request retrieves all of it.

FIX Put the route behind a session check, and return only the caller's own records. Staff scopes read the wider set; nothing returns the full store.

P0 S-02 Any caller can run any privileged domain action

WHERE `app/api/rm/route.ts:15` · `app/lib/rm-db.ts:424`

IMPACT **POST /api/rm** dispatches straight into the action switch with no identity check, and `actorId` is taken from the request body — so the audit ledger records whoever the caller claims to be. An anonymous request can clear its own KYC to Tier 3, freeze another customer, rewrite global pricing, mark a sell-back *DISBURSED*, dispatch an order, or call `resetDemo` to wipe the store.

FIX Derive the actor from a signed server session, never the body. Gate each action behind a required role, and check resource ownership before mutating.

P0 S-03 Sign-in requires only an email address

WHERE `app/lib/rm-db.ts:444` · `app/auth/auth-clients.tsx:74`

IMPACT The `signIn` action looks up a user by email and returns it. There is no password, no OTP, no magic-link token, no verification of any kind. Typing `admin@rockwell.demo` yields a `SUPER_ADMIN` session and a redirect into the ops console.

FIX Add a real credential step. Given the design intent, WebAuthn passkeys plus an email magic link is the right shape — the UI already promises both.

P0 S-04 Sessions are unsigned client-side JSON that carry the role

WHERE `app/lib/use-rm.ts:19-33`

IMPACT The session is `localStorage["rm-session"]` holding `{userId, name, email, role}` as plain JSON. Editing `role` to `"SUPER_ADMIN"` in devtools is a complete privilege escalation, because every gate in the app reads its role from this object.

FIX Move to an `httpOnly`, signed session cookie. The client should never hold an authority claim it can edit.

P0 S-05 Admin routes have no server-side protection**WHERE** `app/admin/ops-shell.tsx:88 · no middleware.ts`**IMPACT** All seventeen `/admin/*` routes are ordinary pages. The only gate is a React early-return inside *OpsShell*, which runs after the server has already rendered and shipped the page. The role it tests comes from the `localStorage` session in S-04.**FIX** Add `middleware.ts` matching `/admin/:path*` and validate the session cookie there, before the route renders.**P0 S-06 A URL mints a session for any seeded account****WHERE** `app/auth/demo/page.tsx:17-24`**IMPACT** `/auth/demo?as=&to;=` signs the visitor in as that user and redirects. It is linked directly from the admin gate as five one-click staff logins. No credential is involved at any point.**FIX** Delete the route, or fence it behind an explicit non-production environment flag.**P0 S-07 Unauthenticated write to the live product catalog****WHERE** `app/api/deploy-catalog/route.ts:107`**IMPACT** **POST** `/api/deploy-catalog` takes `body.items` and appends them to `app/data/products.json` — the live catalog — with no authentication and no schema validation. Attacker-supplied titles, descriptions, prices and image URLs go straight onto the storefront. The same handler also parses and rewrites a 78 MB file synchronously inside the request.**FIX** Require a staff session, validate each item against a schema, and move the write to a background job.**P1 S-08 Unauthenticated routes spawn Python and return raw stderr****WHERE** `app/api/match-jm/route.ts:18 · app/api/synthesize-catalog/route.ts:18`**IMPACT** Both routes `spawn('python3', [script])` on an unauthenticated POST, pipe the request body to the child's stdin, and on failure return the child's raw **stderr** to the caller — leaking absolute filesystem paths, environment detail and tracebacks. Neither has a timeout, so a hung child holds the request open indefinitely.**FIX** Authenticate, return a generic error and log the detail server-side, and add a kill timeout.**P1 S-09 Signed-out /orders lists every customer's orders****WHERE** `app/orders/orders-clients.tsx:52`**IMPACT** `filter(o => !session || o.userId === session.userId)` — when there is no session the predicate is short-circuited to true, so the page renders the full order book: other people's names, items, totals and custody. The banner calls them 'demo transactions'; they are real records from the shared store.**FIX** Invert it: render the empty state and a sign-in prompt when there is no session.

P1 **S-10 Signed-out users are shown a specific customer's private records**

WHERE	<code>vault-sub-clients.tsx:25,463,545</code> · <code>tax-center/tax-client.tsx:14</code>
IMPACT	Four surfaces fall back to <code>uid = session?.userId ?? "u-adrian"</code> . Signed out, <code>/vault/sell-back</code> , <code>/vault/vaultplan</code> , <code>/vault/rewards</code> and <code>/tax-center</code> render Adrian Reyes's holdings, auto-invest plan and tax ledger as if they were the visitor's.
FIX	Remove the fallback; require a session on every account surface.

06 Broken behaviour

Things that do not work when a user follows them. Each was traced to a specific line, not inferred.

P0 B-01 The market floor never reaches the catalog

WHERE `app/market/market-client.tsx:150-360`

IMPACT The primary nav destination is a static prototype: fourteen hardcoded tiles using local `/public/assets` images, every one linking to `href="/product"` — a single fixed Gold Buffalo demo page. None of the 25,457 real products are reachable from the page the site sends customers to first.

FIX Rebuild `/market` on the same server-rendered component the metal pages use, and retire `/product` in favour of `/product/[id]`.

P0 B-02 The Weight filter on /market returns zero results

WHERE `market-client.tsx:151` emits `data-wclassName` · `app/lib/market.js:133` reads `data-wclass`

IMPACT A class-to-className replacement corrupted the data attribute: the markup writes `data-wclassName` on all fourteen tiles while the runtime reads `data-wclass`. Every tile therefore resolves to `weightClass: null`, so clicking `1 oz` or `Fractional` matches nothing and drops the user into the empty state.

FIX Rename the attribute back to `data-wclass` on all fourteen tiles. One-line fix per tile.

P0 B-03 A completed purchase never appears in the vault

WHERE `app/vault/vault-client.tsx:38-90`

IMPACT The vault dashboard is hardcoded — `$28,098, 11.00 oz, 11 coins · 4 types`, a fixed P/L — and never calls `/api/rm`. The order pipeline does its job: `allocateOrder` mints real holdings against the user server-side. Nothing in the customer's vault reflects them.

FIX Rewrite the dashboard against `useRm`, deriving totals from the user's own holdings — the vault sub-pages already do exactly this and can be followed directly.

P0 B-04 Two checkouts exist; one of them persists nothing

WHERE `app/components/order-modal.tsx:69-76`

IMPACT `OrderModal` — opened from the vault CTAs and the global provider — is a second, parallel settlement flow with its own 120-second timer, its own volume tiers and its own surcharges. On execute it fabricates a transaction hash and a serial number, waits 1,600 ms and displays 'confirmed'. It never calls `rmAction`. No order, no holding, no audit entry, no money.

FIX Delete the modal's settlement path and route its CTAs into `createLock` → `/checkout/[lock_id]`, the flow that actually works.

P1 B-05 The multi-angle gallery shows the same photo four times**WHERE** `app/product/[id]/page.tsx:66-104`**IMPACT** Zero of the 25,457 products carry an `images[]` array, so every PDP takes the fallback branch, which renders `p.image` four times under the labels *Obverse*, *Reverse*, *Macro · surface detail* and *In allocated Rockwell vault*. Four identical thumbnails, four different claims about what they show.**FIX** Until the pipeline populates galleries, render a single image and drop the invented view labels.**P1 B-06 Half the catalog is mislabelled and unreachable by filter****WHERE** `app/data/catalog.ts:86-98`**IMPACT** `availOf()` handles nine badge values. The data contains fourteen. The six it does not know — *Vault Intake* (7,019), *Proof Strike* (3,025), *Ingot* (1,144), *Sovereign* (1,113), *Perfect MS70* (745) and *Rare Pick* (4) — hit the default case: **13,050 products, 51% of the catalog**, tagged 'Call for price' directly beside their actual price, and bucketed `key:"other"`, which matches none of the five availability filter buttons.**FIX** Map the six missing badges, and separate the two concepts the badge is currently overloading — availability and product classification.**P1 B-07 216 copper products exist in the data and nowhere in the UI****WHERE** `app/data/catalog.ts:22`**IMPACT** The `Product` union is `gold|silver|platinum|palladium|other`. The catalog contains 216 products with `metal: "copper"`, which is not in the union and has no metal page. They are reachable only by direct `/product/[id]` link.**FIX** Add copper to the union and give it a route, or reclassify the 216 as `"other"`.**P0 B-08 Every product image is hotlinked from two competitors****WHERE** `app/data/products.json · data/rockwell_image_sync_state.db`**IMPACT** The ingestion pipeline has never run. Its state database contains **zero rows** and `rockwell_supabase_image_map.json` reports **unmatched: 25,457**, **primary_updated: 0**. All live images resolve to **sdbullion.com** (13,034) and **images-apmex.com** (12,411). Either origin can rate-limit, hotlink-block or swap the asset at any time, and every product page on the site would break at once.**FIX** Run the pipeline — but resolve B-09 first, because the storage key is derived from the SKU.**P1 B-09 2,408 duplicate SKUs will collide in the image pipeline****WHERE** `app/data/products.json · scripts/rockwell_image_pipeline.py`**IMPACT** The catalog has 25,457 unique ids but only 23,049 unique SKUs. The pipeline writes to `products/{slugify(sku)}/{view}_{hash}.webp` and the linker builds its primary lookup as `sku_map[sku]`. Duplicates share a storage prefix and a map entry, so galleries cross-link between unrelated products and one product's primary image silently becomes another's.**FIX** Key both the storage path and the map on the product id, which is unique, rather than the SKU.

07 Placeholder & filler content

Content that is invented but presented as real. Ordered by how much exposure it carries, not by how visible it is.

P0 F-01 Product reviews, ratings and buyer counts are generated

WHERE `app/data/pdp-fill.ts:29-66`

IMPACT Every PDP displays a star rating seeded to 4.6–5.0, a 'verified buyers' count seeded to 180–2,980, a 'sold this month' figure seeded to 90–4,290, and three reviews drawn from a pool of six invented testimonials attributed to handles like `@stackmore` and `@ledger_larry`. They are deterministic per product id, so they are stable across reloads and read as genuine.

FIX Remove them before any public launch. Fabricated reviews and invented 'verified buyer' counts are directly actionable under the FTC's consumer-review rule; this is the highest-liability item in the codebase.

P0 F-02 The tax centre issues a 1099-B with a fabricated basis

WHERE `app/tax-center/tax-client.tsx:26-33,44-50`

IMPACT The CSV export is headed *'Form 1099-B · Rockwell Metals LLC · TIN 88-2044417'*. Cost basis for every disposal is computed as `proceeds x 0.955` and realised gain as a flat `proceeds x 0.045` — a fixed 4.5% gain on every sale regardless of what the customer actually paid, even though the true basis is on the order record. The 'Form 8300 events · auto-filed' tile counts any purchase over \$10k, wires included, and files nothing.

FIX Compute basis from the actual acquisition orders. Remove the TIN and the 1099-B framing until a real filing integration exists, and correct the 8300 rule to cash and cash-equivalents only.

P0 F-03 Identity verification is an animation

WHERE `app/auth/auth-clients.tsx:262-273` · `app/lib/rm-db.ts:456`

IMPACT The KYC screen steps through six labels on a 600 ms timer, then calls `kycSubmit`, which sets `kycStatus: "CLEARED"` and `kycTier: "TIER_2"` unconditionally. No document is ever captured. The audit entry it writes claims `engine: "Stripe Identity · liveness pass · risk 04/100"` — an evidence record for a check that did not happen.

FIX Wire a real provider, or gate the screen behind a demo flag. The false audit entry should go either way.

P1 F-04 Passkey, Google SSO and hardware-key sign-in are decorative

WHERE `app/auth/auth-clients.tsx:132-152`

IMPACT All three buttons call the same `signIn(method)`, which does an email lookup and ignores the method entirely. The distinct spinner text — 'Verifying passkey...', 'Checking security key...' — implies a verification that never occurs.

FIX Implement WebAuthn for the passkey and hardware-key paths, or remove the buttons.

P1 F-05 Spot price is a literal in twelve files**WHERE** 29 sites across 12 files · `app/lib/checkout.ts:52`**IMPACT** \$2,387.40 / +0.42% is hardcoded across the home page, PDP, footer, command palette, catalog header, price-alert modal and ops topbar. More consequentially, `createLock` writes `spotAtLock: 2387.4` into every order, and the pricing engine computes its preview ladder off the same constant — so premium and margin maths is anchored to a fictional mark.**FIX** Introduce one server-side spot source, cache it, and read every display and every lock from it.**P1 F-06 Category pages are banded 'Placeholder inventory · internal test data'****WHERE** `app/components/catalog-page.tsx:59,163`**IMPACT** The eyebrow above the title reads *'Placeholder inventory · internal test data'* and the filter rail closes with *'Placeholder catalog for internal testing. Prices are static snapshot values from the source data.'* Both are live on `/gold`, `/silver`, `/platinum`, `/best-sellers`, `/new-arrivals`, `/on-sale` and every mint and series page beneath them.**FIX** Remove both strings. The second is also the only honest statement on the page about pricing, which is worth resolving properly rather than deleting.**P1 F-07 Twenty-six legal and informational links go nowhere****WHERE** `home-client.tsx:720-738` · `product-client.tsx:500-516` · `site-footer.tsx:48` · `auth-clients.tsx:221` · `market-client.tsx:402-418` · `vault-client.tsx:832-850`**IMPACT** About, Authentication, Custody, Terms, Privacy and Disclosures are `href="#"` in six files. So is the *custody charter* link inside the sign-up consent checkbox — the document a user must affirmatively agree to in order to open an account cannot be read.**FIX** The charter link is the blocking one; a consent checkbox pointing at nothing is not enforceable consent.**P1 F-08 Unrealised vault P/L is a fixed +4% by construction****WHERE** `app/vault/vault-sub-clients.tsx:15-19`**IMPACT** `LIVE_UNIT` is a seven-entry hardcoded price map covering the original demo coins. Every other holding falls through to `Math.round(h.costUsd * 1.04)`, so its unrealised gain is 4% of cost by definition. Any real purchase lands in the fallback.**FIX** Value holdings from the same spot source as F-05, applied to weight and purity.**P2 F-09 The crypto QR code encodes nothing****WHERE** `app/checkout/[lock_id]/checkout-client.tsx:20-56`**IMPACT** `VectorQrCode` is a hand-drawn SVG — three finder patterns and decorative rectangles — presented beside 'Send exactly \$X in USDC' and the address `0x7f4E...rockwell-settlement.eth`. Scanning it produces nothing. The wire panel likewise shows a fixed ABA routing number, `0260-0959-3`, on every order.**FIX** Generate a real payment-URI QR at request time, or remove the code and show the address alone.

P2 **F-10** **A legacy demo product page is still routable**

- WHERE** `app/product/page.tsx` · `app/product/product-client.tsx`
- IMPACT** `/product` renders a 538-line hardcoded Gold Buffalo page with its own spot ticker and its own footer. It is the destination of all fourteen market tiles (B-01) and is separately reachable by URL.
- FIX** Delete once `/market` is rebuilt against real products.

08 Performance

Two findings here are severe enough to make the site unusable on anything but a fast desktop connection.

P0 P-01 The 78 MB product catalog is shipped to the browser on every page

WHERE `app/components/command-palette.tsx:1-4 · app/data/catalog.ts:1`

IMPACT `command-palette.tsx` is a client component and imports `{ products }` from `catalog.ts`, which does `import productsJson from "../products.json"`. That pulls the entire **78 MB** catalog into a client chunk. The palette is mounted globally by `GlobalFintechProvider` in the root layout, so the chunk is requested on **every route**, including the homepage. This is, by a wide margin, the most expensive line in the codebase.

FIX Move search behind an API route or a prebuilt index of `id / title / sku / mint` — roughly 2 MB — and keep the full catalog on the server.

P1 P-02 Command palette scans 25,457 products on every keystroke

WHERE `app/components/command-palette.tsx:16-26`

IMPACT Each keystroke runs an unmemoised, undebounced linear filter calling `toLowerCase()` on four fields per product — about **100,000 transient strings per key** — and then discards all but six results. On a mid-range laptop this is visible input lag; on mobile it is unusable.

FIX Debounce to ~150 ms, lowercase the index once at build time, and cap the scan.

P0 P-03 Category pages render the entire catalog into one document

WHERE `app/components/catalog-page.tsx:170 · hidden={i >= 20}`

IMPACT Every product in a category is server-rendered as a full tile, then all but the first twenty are hidden with the `hidden` attribute and paged through in the DOM. `/silver` emits **14,829 tiles** $\approx$ **8.4 MB of HTML**; `/gold` $\approx$ 5.1 MB. The browser parses and lays out all of it to show twenty.

FIX Paginate on the server via `searchParams`, or virtualise the grid. This single change is worth more than every other performance fix combined on those routes.

P1 P-04 Every poll transfers the whole database

WHERE `app/lib/use-rm.ts:73-88`

IMPACT `useRm` polls `GET /api/rm` every 4–6 seconds and each response is the complete store — all users, orders, holdings, shipments, tickets and the audit ledger. Several pages call `useRm()` more than once, and each call runs its own interval.

FIX Split into per-resource endpoints scoped to the caller, add `ETag/304`, and prefer SSE for the live queues.

P1 P-05 The JSON store has no write locking and no serverless story**WHERE** `app/lib/rm-db.ts:330-356`**IMPACT** `getDb()` caches the store in a module-level variable and `saveDb()` serialises and rewrites the whole file on **every** mutation, with no lock. Two concurrent writes lose one. On any serverless host the module cache is per-instance, so state silently diverges between lambdas and the filesystem is read-only in the first place.**FIX** This is the intended Supabase/Postgres cutover. Until then it is a single-process development store and should not be deployed.**P1 P-06 59 raw tags, no next/image, no image config****WHERE** `59 sites across the app · next.config.ts is empty`**IMPACT** No responsive `srcset`, no modern format negotiation, no lazy-loading policy beyond a manual `loading="lazy"` on catalog tiles, and no intrinsic dimensions on most — so every image causes layout shift. Source assets are full-resolution APMEEX and SD JPEGs, up to 1.4 MB each in the pipeline's own test sample, rendered into roughly 200 px tiles.**FIX** Adopt `next/image` with the remote patterns configured, once B-08 moves images onto owned storage.**P2 P-07 Three unrelated CSS bundles load on every page****WHERE** `app/layout.tsx:1-2 · app/*.css`**IMPACT** `styles.css` (60 KB) and `rm.css` (25 KB) are imported globally, with `market.css`, `product.css` and `vault.css` layered per route — 130 KB of uncompressed CSS, essentially none of it scoped or tree-shaken. Tailwind 4 is installed and unused.**FIX** Either commit to the hand-written CSS and split it per route, or migrate to Tailwind. Carrying both costs a dependency and gains nothing.**P2 P-08 Roughly 400 MB of data files sit untracked in the repo****WHERE** `data/ · app/data/ · .gitignore`**IMPACT** `jmbullion_all_product_data.csv` 63 MB, `catalog_studio_checkpoint.json` 39 MB, `rockwell_catalog_progress.db` 25 MB, `products.json` 78 MB, plus two `.bak` copies of the catalog at 117 MB combined. Only `data/db.json` is gitignored. Committing this makes the repository permanently large.**FIX** Gitignore the intermediate datasets, keep the two catalog backups out of version control, and treat `products.json` as a build artefact produced by `build:data`.**P2 P-09 The project cannot currently be built or run****WHERE** `node_modules/ is empty`**IMPACT** No dependencies are installed, so `next build`, `next dev` and `tsc --noEmit` all fail. Nothing in this audit was verified at runtime, and the compiler has not checked the codebase. AGENTS.md's instruction to consult `node_modules/next/dist/docs/` cannot be followed either.**FIX** Run `npm install`, then `tsc --noEmit` and `next build` — expect further findings from both.

09 Standards & engineering quality

P1 T-01 No error, loading or not-found boundaries anywhere

WHERE `no error.tsx` / `global-error.tsx` / `not-found.tsx` / `loading.tsx` in `app/`

IMPACT Any thrown error in a server component drops the user on the framework's default error screen. `notFound()` in the PDP renders Next's stock 404 rather than anything Rockwell-branded. The catalog pages, which are the slowest in the app, have no streaming loading state.

FIX Add `error.tsx`, `not-found.tsx` and route-level `loading.tsx`.

P1 T-02 No SEO surface on a 25,000-page storefront

WHERE `no sitemap.ts` / `robots.ts` / `opengraph-image` / `JSON-LD`

IMPACT There is no sitemap, so 25,457 product pages have no discovery path; no robots policy; no OG image, so every share renders bare; and no `Product` JSON-LD, so no rich results — price, availability and rating never reach the SERP. For a catalogue this size that is the single largest missed channel.

FIX Add `app/sitemap.ts` (chunked at 50k URLs), `app/robots.ts`, an OG image, and `Product` + `BreadcrumbList` JSON-LD on the PDP.

P1 T-03 Dark mode is erased on every page load

WHERE `app/layout.tsx:5-7`

IMPACT The inline theme script runs `localStorage.setItem("rm-theme", "light")` unconditionally before `ThemeToggle`'s effect reads it. The toggle works until the user navigates, at which point their choice is overwritten. The site ships a dark theme that cannot be kept.

FIX Have the inline script *read* the stored preference and fall back to `prefers-color-scheme`, never write it.

P1 T-04 Modal accessibility is incomplete

WHERE `app/components/*-modal.tsx` · 6 components

IMPACT All six set `role="dialog"` and `aria-modal="true"`, and correctly stop propagation on the panel. None trap focus, none restore focus on close, none close on Escape, and none carry `aria-labelledby`. A keyboard user can tab out of an open dialog into the page behind it, and a screen reader announces an unnamed dialog.

FIX One shared `<Modal>` wrapper with a focus trap, Escape handling, focus restore and a labelled heading fixes all six at once.

P2 T-05 Admin table rows are not keyboard accessible

WHERE `orders-admin-clients.tsx:48` · `desk-clients.tsx:96`

IMPACT Rows navigate via `onClick={() => window.location.href = ...}` on a bare `<tr>`: no tab stop, no focus ring, no Enter handling — and a full document reload instead of a client transition, which discards the polled state and re-downloads the bundle.

FIX Wrap the primary cell in a `<Link>`, or give the row `tabIndex`, a key handler and a router push.

P2 T-06 window.confirm() guards irreversible operations**WHERE** `orders-admin-clients.tsx:79` · `desk-clients.tsx:20`**IMPACT** Sell-back disbursement, order dispatch and customer freeze are all confirmed with a native `window.confirm`. It is unstyled, blocks the main thread, cannot show the context of what is being confirmed, and freezes any browser-automation session outright.**FIX** Use an in-app confirmation dialog that restates the amount, the customer and the consequence.**P2 T-07 Two runtimes fight over document.body.className****WHERE** `catalog-runtime.tsx:10` · `pdp-runtime.tsx:26` · `market-client.tsx:10` · `vault-client.tsx:13` · `product-client.tsx`**IMPACT** Five client components set `document.body.className` on mount and clear it on unmount. During a client-side transition the incoming component's effect can run before the outgoing cleanup, so the body class ends up empty and the page renders unstyled until a reload.**FIX** Set the class from the server layout per route segment rather than imperatively on mount.**P2 T-08 2,900 lines of ES5 DOM script shadow the React tree****WHERE** `app/lib/{home,market,vault,product,catalog,pdp}.js`**IMPACT** Six IIFE-style scripts drive filtering, sorting, pagination, tickers, galleries and the price-lock timer by direct DOM mutation, in parallel with React's own state. They are untyped, untestable, duplicated across files, and the source of B-02.**FIX** Port to React state incrementally, starting with `catalog.js`, which backs the six live category routes.**P3 T-09 No tests, and three images without alt text****WHERE** `package.json` · `home-client.tsx:85,532` · `product-client.tsx:40`**IMPACT** There is no test runner, no spec file and no CI. For a system whose correctness claim rests on an eight-state order machine and serial-level custody, the transition layer in `rm-db.ts` is the obvious first thing to cover. Separately, three `<img>` tags carry no `alt`, and no page offers a skip-to-content link.**FIX** Add Vitest and cover `runAction` transitions first — it is pure, synchronous and the highest-value surface in the codebase.

10 Catalog data quality

Measured directly against `app/data/products.json` — 25,457 records.

MEASURE	VALUE	READING
Total products	25,457	Complete descriptive copy on all records — no empty summaries
Unique ids	25,457	Clean
Unique SKUs	23,049	2,408 duplicates — collides in the image pipeline (B-09)
Metals	Ag 14,829 · Au 9,090 · other 750 · Pt 491 · Cu 216 · Pd 81	Copper is not in the type union and has no route (B-07)
Badge values	14 distinct	availOf() handles 9; the other 5 cover 13,050 products (B-06)
Products with no price	3,357 (13.2%)	Render as an em-dash; no 'enquire' path exists
Products with no year	7,226 (28.4%)	Excluded from year sort and the new-arrivals rule
Products with a gallery	0	PDP shows the same photo four times (B-05)
Images on own storage	0	All 25,457 hotlink SD Bullion or APMEX (B-08)
Best sellers	79	Only 'Top Pick' qualifies — a thin landing page
New arrivals	69 + year ≥ 2026	'New', 'Pre-Sale', 'Coming Soon' are rare in the data
On sale	1,422	Healthy

The catalog itself is the platform's strongest asset

Every one of the 25,457 records carries a real short summary, a full description, metal content, purity, grade, diameter, thickness, face value, IRA eligibility and separate obverse and reverse descriptions. That is a genuinely valuable dataset and it is already complete. The problems above are all in how the application reads it — badge mapping, type union, image hosting — not in the data.

11 Remediation roadmap

Sequenced so that each stage unblocks the next. Effort is engineering days for one developer familiar with the codebase.

Stage 1 — Close the doors · ~4 days

#	ACTION	UNBLOCKS
1	Signed httpOnly session cookie; remove role from client storage	S-03, S-04
2	middleware.ts gating /admin/* and every account route	S-05
3	Session + role check on /api/rm; derive actorId server-side; scope GET to the caller	S-01, S-02
4	Auth on the three catalog API routes; generic errors; child-process timeout	S-07, S-08
5	Delete /auth/demo; remove the four u-adrian fallbacks; fix the /orders predicate	S-06, S-09, S-10

Stage 2 — Make the journey real · ~6 days

#	ACTION	UNBLOCKS
6	Rebuild /market on the server-rendered catalog component; retire /product	B-01, B-02, F-10
7	Rewrite the /vault dashboard against useRm, following the sub-pages	B-03
8	Route OrderModal into the real price-lock flow; delete its settlement path	B-04
9	Map the six missing badges; add copper to the union and a route	B-06, B-07
10	Single server-side spot source feeding display, locks and pricing	F-05, F-08

Stage 3 — Remove the fiction · ~2 days

#	ACTION	UNBLOCKS
11	Strip generated reviews, ratings and buyer counts from the PDP	F-01
12	Compute tax basis from real orders; drop the TIN and 1099-B framing; fix the 8300 rule	F-02
13	Gate or implement KYC; remove the false audit evidence string	F-03
14	Write the custody charter, terms, privacy and disclosures; wire all 26 links	F-07
15	Remove the 'Placeholder inventory' banding; remove or implement the SSO buttons	F-04, F-06

Stage 4 — Make it fast · ~5 days

#	ACTION	UNBLOCKS
16	Move palette search server-side or onto a slim index; drop the catalog from the bundle	P-01, P-02

#	ACTION	UNBLOCKS
17	Server-side pagination on the six category routes	P-03
18	Per-resource scoped endpoints with ETag; SSE for the ops queues	P-04
19	Fix the image pipeline SKU collision, then run it; adopt next/image	B-08, B-09, P-06
20	Gitignore the intermediate datasets; treat products.json as a build artefact	P-08

Stage 5 — Meet the bar · ~4 days

#	ACTION	UNBLOCKS
21	npm install; clear tsc --noEmit and next build; triage what they surface	P-09
22	Error, not-found and loading boundaries across the app	T-01
23	sitemap.ts, robots.ts, OG image, Product + BreadcrumbList JSON-LD	T-02
24	Shared accessible Modal wrapper; fix the theme script; keyboard-accessible admin rows	T-03, T-04, T-05
25	Vitest over runAction transitions; port catalog.js to React state	T-08, T-09

Closing note

Roughly three engineering weeks separates this from a defensible production launch, and the bulk of that is not new construction — it is connecting a well-built system to the pages customers actually see, and deleting the scaffolding that stands in for it. The domain layer, the order machine, the custody model and the ops console are real work worth keeping. Stage 1 should not wait on the rest; as the application stands, deploying it publicly would expose every customer record it holds.